Architecture & Design Review Board (ARB) Governance Framework for a Workspace Transformation

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

Contents

Executive Summary — Architecture & Design Review Board (ARB) Governance Framework 5

What you get . . . 5

What this delivers . . . 5

The standard deployment model . . . 6

Outcome . . . 6

1. Solution Overview — Architecture & Design Review Board (ARB) Governance Framework 6

The problem this solves . . . 6

What an ARB is (and is not) . . . 6

Relationship to the target architecture . . . 7

The five review gates . . . 7

The decision-record discipline . . . 8

Design-drift prevention . . . 8

The governance KPI set . . . 8

What this solution does and does not do . . . 8

The standard deployment model . . . 8

2. Requirements Guide — Architecture & Design Review Board (ARB) Governance Framework

2.1 Executive sponsorship and a mandate . . . 9

2.2 A ratified target architecture (the approved standards baseline) . . . 9

2.3 Board membership and decision rights defined . . . 9

2.4 The vendor/enterprise authority boundary . . . 9

2.5 A tracking tool for submissions and decisions . . . 10

2.6 Microsoft Graph access for the board's tooling (read-only for reads) . . . 10

2.7 A meeting cadence and secretariat . . . 10

Checklist . . . 10

Global Requirements — Google Workspace & Gemini Solutions 11

1. Google Workspace / Cloud Identity tenant . . . 11

2. Source Microsoft 365 / Entra environment (for migration & coexistence) . . . 11

3. DNS control for the routed domains . . . 12

4. Identity & provisioning source of truth . . . 12

5. A pilot scope (OU / group) and change control . . . 12

6. Security, compliance & governance stakeholders . . . 12

3. Implementation Guide — Architecture & Design Review Board (ARB) Governance Framework 13

Step 1 — Assess: confirm the mandate and the baseline . . . 13

Step 2 — Configure: write the charter and decision rights . . . 14

Step 3 — Configure: define the five review gates and their templates . . . 14
Step 4 — Configure: stand up the tracker and the register . . . 14
Step 5 — Pilot: run one governance cycle on a pilot workstream . . . 14
Step 6 — Run the drift-prevention loop on the pilot . . . 15
Step 7 — Validate: the pilot gate (GATE) . . . 15
Step 8 — Expand: govern every workstream . . . 15
5. Testing Guide — Architecture & Design Review Board (ARB) Governance Framework 16
Test 1 — Charter and decision rights are unambiguous . . . 16
Test 2 — Membership groups exist and match the roster . . . 16
Test 3 — Every gate has a submission template and checklist . . . 16
Test 4 — Submissions are inventoried and classified . . . 16
Test 5 — Conformance is checked against the approved baseline . . . 16
Test 6 — Every decision has a record . . . 17
Test 7 — The Gap Register captures all open drift . . . 17
Test 8 — Remediation opens owned, dated actions (safely) . . . 17
Test 9 — Governance is not the bottleneck . . . 17
Pass/fail gate . . . 17
6. Troubleshooting Guide — Architecture & Design Review Board (ARB) Governance Framework 17
The board's decisions are ignored . . . 17
Governance has become the bottleneck . . . 18
Reviewers disagree and decisions stall on ties . . . 18
`GWARB-Validation.ps1` flags everything as No-Baseline . . . 18
Designs appear with a gate outside the standard five . . . 18
Decisions are made but not recorded . . . 18
Approved exceptions are quietly becoming the norm . . . 18
`Connect-MgGraph` fails or the tracker list cannot be read . . . 19
`GWARB-Remediation.ps1` or `GWARB-Set-ScopeTargeting.ps1` made no changes . . . 19
The board outlives its usefulness / becomes a rubber stamp . . . 19
7. Best Practices — Architecture & Design Review Board (ARB) Governance Framework 19
Authority . . . 19
Decision rights . . . 20
Gates and cadence . . . 20
Decision records . . . 20
Drift prevention . . . 20
Scope discipline . . . 21

8. Deliverables Summary — Architecture & Design Review Board (ARB) Governance Framework 21
Documentation . . . 21
Named artifacts produced . . . 21
Design-Review-Board runbook (outline) . . . 22
Templates outline (what ships to be filled in) . . . 22
PowerShell scripts . . . 23
What the buyer receives . . . 23
Outcome . . . 23
Appendix A — PowerShell Scripts 24

Executive Summary — Architecture & Design Review Board (ARB) Governance Framework

A large Google Workspace + Gemini transformation is designed by many hands — internal workstream leads, a migration partner, identity and security specialists, third-party integrators. Every day they make design decisions: the identity and sync model, the migration wave patterns, the security configuration, the work-transformation and Gemini adoption approaches. Without a forum that has the authority to approve those designs and stop the bad ones, two things happen. The estate drifts from the target architecture until nobody designed what you end up with — and the party doing the work quietly decides the architecture, optimizing for delivery speed rather than your standards, security baseline, and lock-in posture.

This is a complete, repeatable package for standing up an **Architecture & Design Review Board (ARB)** with real decision authority over a Google Workspace + Gemini program. It gives a senior architecture owner a charter, defined decision rights, five reusable review gates, submission and decision-record templates, a design-drift-prevention loop, and the scripts and KPIs to run it — so every major design is reviewed against the approved target architecture and vendor execution stays aligned to enterprise standards.

What you get

• A plain-language Solution Overview of what an ARB is, how it relates to the target architecture, the five review gates, the decision-record discipline, and how drift is detected and stopped.

• A Requirements Guide with verify / implement-if-missing for every prerequisite — sponsorship, a ratified standards baseline, membership and decision rights, the vendor/enterprise authority boundary, the tracker, and the meeting cadence.

• An Implementation Guide of precise, ordered steps — confirm the mandate and baseline, write the charter and decision rights, open the five gates with their templates, stand up the register, then pilot on one workstream, validate, and expand program-wide.

• Five PowerShell scripts for design/decision discovery, conformance validation against the approved standards, a decision/gap register with governance KPIs, safe remediation-action tracking, and an ARB membership-group builder.

• A Design-Review-Board runbook and a full template set (charter, decision-rights matrix, per-gate submission templates and checklists, and the decision-record template), plus Testing and Troubleshooting guides with explicit gates and field-proven Best Practices.

What this delivers

A governed program of record: an ARB `Charter - Transformation - Ratified`, a `Decision Rights Matrix

• ARB - Program, `five review gates with templates, an authoritative` Decision Register -

Transformation ARB - Program-Wide, a rolling Gap Register - Design Drift - Rolling, and an ARB KPI Set - Governance - Standard`. Proven on a pilot workstream before it governs the whole program.

The standard deployment model

Assess to Configure to Pilot to Validate to Expand — with a pilot gate that must pass before the board's remit is expanded program-wide.

Outcome

Every major design decision reviewed and approved against the target architecture — no unmanaged vendor-driven drift — with a governance forum that has real authority, stood up by a single architecture owner and validated on a pilot before it scales.

1. Solution Overview — Architecture & Design Review Board (ARB) Governance Framework

This overview is the one place that explains *why* a transformation needs a design review board with real authority, and *how* that board relates to the target architecture and the delivery workstreams. The rest of the package is procedure and templates; this section is the operating model you carry into them.

The problem this solves

A large Google Workspace + Gemini transformation is executed by many hands — internal workstream leads, a migration partner, identity and security specialists, and third-party integrators. Each makes design decisions daily: how the identity/sync model is shaped, which migration wave pattern is used, how a security control is configured, how a work-transformation pattern is rolled out. Without a forum that has the authority to *approve* those designs and *stop* the bad ones, two failure modes recur:

- **Unmanaged drift** — the estate slowly diverges from the target architecture. No single decision looks wrong, but the sum is an environment nobody designed, that is hard to secure, support, or audit.
- **Vendor-led architecture** — the party doing the work also decides the architecture by default, optimizing for delivery speed rather than the enterprise's long-term standards, lock-in posture, or security baseline.

An Architecture & Design Review Board (ARB) replaces both with a known authority: every major design is submitted, reviewed against the approved target architecture, and formally approved, conditionally approved, or rejected — with the decision recorded. Delivery stays fast, but architecture stays owned by the enterprise.

What an ARB is (and is not)

An ARB is a standing governance forum with **defined decision rights**: a chartered membership, a quorum, a set of review gates, and the authority to approve or block designs at those gates. It is the enterprise's architectural conscience for the program.

- It **is**: a decision authority, a conformance check against the target architecture, a system of

record for design decisions, and a drift-detection mechanism.

- It **is not**: a project management office (it governs designs, not schedules), a rubber stamp (a gate that always approves is not a gate), or a replacement for the workstreams' own engineering (it reviews their designs; it does not do their work).

The board's authority is real precisely because the charter draws a hard line: **the enterprise owns the architecture; the vendor executes within it.** Vendors and workstreams retain full latitude on *how* they deliver, but the *what* — the architectural standards, the identity model, the security baseline, the migration patterns — is approved by the board.

Relationship to the target architecture

An ARB is only as strong as the standard it governs against. This framework assumes a ratified **target architecture** for the Google Workspace + Gemini estate — the identity and provisioning model, the security and DLP baseline, the collaboration and data-residency decisions, the Gemini adoption posture, and the migration patterns. That target architecture is the **approved standards baseline** the board measures every submission against. Where your program has a separate target-architecture blueprint, the ARB is the governance body that enforces it; where it does not, the board's first act is to ratify one, because a review gate with no standard behind it cannot make a defensible decision.

The five review gates

Major designs pass through the board at one of five standard gates. Each gate has a submission template, a review checklist, and a decision record. The gates are the reusable spine of the whole framework — name them and reuse them verbatim:

Gate	What it governs	Typical submitter
Migration Wave Plan	Scope, sequencing, cutover and rollback for a migration wave	Migration workstream / partner
Identity & Sync Model	Source of truth, provisioning (GCDS/SCIM), SSO/federation, coexistence	Identity workstream
Security Configuration	DLP, context-aware access, sharing, admin roles, data residency	Security workstream
Data-Migration Pattern	Mail/Drive/Chat movement approach, fidelity, staging, verification	Migration workstream / partner
Work-Transformation Approach	Collaboration, AppSheet/automation, and Gemini adoption patterns	Change / adoption workstream

A design is submitted to exactly one gate (its primary risk), reviewed against the standards relevant to that gate, and receives one decision: **Approved**, **Approved with conditions** (an approved exception is recorded), or **Rejected** (rework required). Nothing at a gate proceeds to implementation without a decision record.

The decision-record discipline

Every gate decision produces an **Architecture Decision Record (ADR)** in the `Decision Register - Transformation ARB - Program-Wide`: the design, the standards it was measured against, the decision, the rationale, any conditions/exceptions, and the owner. The register is the program's architectural memory — it is what makes a later "why did we do it this way?" answerable, and what makes drift visible when reality diverges from an approved decision.

Design-drift prevention

Approving a design is not the end; reality drifts from approved designs as delivery pressure mounts. The framework's drift-prevention loop is: **discover** what has actually been built (`GWARB-Discovery.ps1`), **validate** it against the approved standards baseline (`GWARB-Validation.ps1`), **report** the gap between approved-and-built into a Gap Register (`GWARB-Report.ps1`), and **remediate** each gap as an owned, dated action (`GWARB-Remediation.ps1`) — either bringing the build back to standard or formally approving the deviation as an exception. Drift that is caught, adjudicated, and recorded is governed; drift that is silent is the failure mode this whole solution exists to prevent.

The governance KPI set

The board's own effectiveness is measured by the `ARB KPI Set - Governance - Standard`:

- **Decisions logged** — every gate outcome has an ADR (target: 100% of gate-stage designs).
- **Approval rate** — approvals vs total decisions, watched for both rubber-stamping (too high) and bottlenecking (too low).
- **Open drift** — non-conformant findings not yet adjudicated (target: trending to zero).
- **Time-in-gate** — average days from submission to decision (watched so governance does not become the bottleneck).
- **Exception count** — approved deviations, tracked so exceptions do not quietly become the norm.

What this solution does and does not do

- It **does**: stand up an ARB with a charter, decision rights, five review gates, submission and decision-record templates, a drift-prevention loop, and the scripts and KPIs to run it.
- It **does not**: design your target architecture for you, make architectural decisions on your behalf, or govern schedule and budget (that is the PMO's remit). It gives the board the authority, the process, and the artifacts; the board makes the calls.

The standard deployment model

Assess to Configure to Pilot to Validate to Expand. You stand up the charter and gates, run a **pilot cycle** on one workstream's designs to prove the board works, validate that decisions are being made and recorded without becoming a bottleneck, then expand the board's remit across every workstream in the

program.

2. Requirements Guide — Architecture & Design Review Board (ARB) Governance Framework

Verify every requirement before you charter the board or open a gate. Each entry follows: Description · Why · Verify · Implement-if-missing. Category-wide prerequisites (Workspace/Cloud Identity tenant, source M365/Entra, DNS control, identity source of truth, pilot scope and change control, security/compliance stakeholders) are in `global-requirements-google.md` — confirm those first; the items below are specific to standing up the ARB.

2.1 Executive sponsorship and a mandate

Description: A named executive sponsor (typically the enterprise architect owner, CIO/CTO delegate, or transformation lead) who charters the board and backs its decision rights. Why: An ARB governs by authority. Without a sponsor whose backing makes "Rejected" stick, the board is advisory only and drift returns. Sponsorship is the single most important prerequisite. Verify: You can name the sponsor and point to a written mandate (email, program charter, or steering minutes) that grants the board decision authority over design. Implement: Secure a sponsor and a one-paragraph mandate before the first board meeting; the mandate is quoted verbatim in the `ARB Charter - Transformation - Ratified`.

2.2 A ratified target architecture (the approved standards baseline)

Description: A ratified target-architecture standard for the Google Workspace + Gemini estate — the identity/sync model, security/DLP baseline, collaboration and data-residency decisions, Gemini adoption posture, and migration patterns — that the board measures submissions against. Why: A review gate with no standard behind it cannot make a defensible decision; conformance is meaningless without a baseline. This is the reference every gate uses. Verify: A target-architecture document (or blueprint solution) exists and its key standards can be expressed as `Standard,ExpectedValue` rows for `GWARB-Validation.ps1`. Implement: If none exists, the board's first ratified act is to adopt one (or the program's target-architecture blueprint) as the baseline. Do not open conformance gates until it is ratified.

2.3 Board membership and decision rights defined

Description: A defined membership — voting members (quorum-bearing decision-makers: enterprise architecture, security, identity, and a business/change representative) and advisory reviewers (subject-matter experts who review but do not vote) — with the quorum and the approve/reject authority written down. Why: Decision rights are the substance of the board. Who votes, what quorum is, and who may approve/reject must be unambiguous, or decisions are challengeable. Verify: You can name the voting members, the quorum number, and the escalation path for a tie or a blocked decision. Implement: Define the roster in the charter and provision the two membership groups with `GWARB-Set-ScopeTargeting.ps1` (`ARB - Voting Members - Board`, `ARB - Advisory Reviewers - Panel`).

2.4 The vendor/enterprise authority boundary

Description: A written boundary stating that the enterprise owns the architecture and approves designs, while the delivery partner/vendor executes within approved designs and submits changes to the board. Why: The most common transformation drift is vendor-led architecture. The boundary is what gives the board authority over a party it does not manage day to day. Verify: The delivery contract or SOW (or a governance addendum) references the ARB's approval rights; the vendor's leads know they submit designs to gates. Implement: Add a governance clause to the SOW/engagement or a signed governance addendum before the partner begins design work; restate the boundary in the charter.

2.5 A tracking tool for submissions and decisions

Description: A controlled place to track submissions, gate state, and decision records — a Microsoft List / SharePoint list, a work-tracker, or a governed spreadsheet — that the scripts can read/export. Why: The board runs on a queue and a register; without a single tracked source, submissions get lost and decisions are not recorded. Verify: A tracker exists with, at minimum, columns for DesignId, Gate, State, Workstream, DecisionRecordId, Submitted, and Owner (the columns `GWARB-Discovery.ps1` reads). Implement: Stand up the tracker (a Microsoft List is convenient because `GWARB-Discovery.ps1` and `GWARB-Remediation.ps1` can read/write it via Microsoft Graph) and the `Decision Register - Transformation ARB - Program-Wide`.

2.6 Microsoft Graph access for the board's tooling (read-only for reads)

Description: The ability to run Microsoft Graph PowerShell with `Sites.Read.All` (to read a Microsoft List tracker) and `Group.ReadWrite.All` + `User.Read.All` (to provision the membership groups). Write scopes (`Sites.ReadWrite.All`) are needed only when the remediation script creates action items. Why: The discovery, report, and remediation scripts read the tracker and manage the ARB groups; reads are least-privilege and read-only. Verify: `Connect-MgGraph -Scopes 'Sites.Read.All'` succeeds and you can read the tracker list; the membership groups can be enumerated. Implement: Install the Microsoft Graph PowerShell SDK (`Install-Module Microsoft.Graph -Scope CurrentUser`) and consent the least-privilege scopes; grant write scopes only to the operator who runs remediation.

2.7 A meeting cadence and secretariat

Description: A regular board cadence (for example, weekly gate reviews with an out-of-band fast-track for urgent designs) and a named secretariat who runs the queue, publishes decisions, and maintains the register. Why: Governance that meets irregularly becomes the schedule bottleneck delivery teams route around. A predictable cadence and an owner keep time-in-gate low. Verify: A recurring meeting exists on the calendar with an agenda template, and a secretariat is named. Implement: Schedule the cadence, appoint the secretariat, and adopt the agenda/decision templates from the Implementation Guide.

Checklist

- Executive sponsor and written mandate secured (2.1)
- Target architecture ratified as the approved standards baseline (2.2)
- Voting members, advisory reviewers, quorum, and escalation defined (2.3)
- Vendor/enterprise authority boundary written into SOW/addendum (2.4)

- Submission/decision tracker and `Decision Register` stood up (2.5)
- Microsoft Graph access confirmed (least-privilege reads) (2.6)
- Board cadence scheduled and secretariat named (2.7)
- Pilot workstream chosen for the first governance cycle (global G5)

Global Requirements — Google Workspace & Gemini Solutions

These requirements apply to **every** solution in the **Google** category. They are the common foundation each Google Workspace / Gemini solution assumes is already in place (or stands up as part of its own Requirements section). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Google solution).*** *The* ***Google Admin console*** *(admin.google.com),* ***Google Cloud / Cloud Identity****, and the* ***Microsoft / Entra*** *sides are authored at exact, current click-by-click detail from the vendors' public documentation.* ***Gemini Enterprise*** *capabilities that are in preview or rolling out are* ***cited or flagged "verify in your tenant"*** *— never invented.* ***Third-party migration tools*** *(e.g. for mail/Drive/Chat data movement) and* ***third-party enterprise apps*** *(ServiceNow, SAP, Databricks, etc.) are integrated at the pattern/architectural level; their internal console steps are deferred to the vendor's own docs.*

1. Google Workspace / Cloud Identity tenant

- **Description:** A provisioned Google Workspace (or Cloud Identity) tenant with a **super-admin** account for configuration, and the appropriate Workspace edition/licensing for the capability in scope (e.g. Enterprise Plus for Vault/CAA/DLP, Gemini/AI add-on where required).
- **Why required:** All configuration, policy, and API work is performed against this tenant.
- **How to verify:** Sign in to **admin.google.com** as a super-admin; confirm edition and license counts under **Billing / Subscriptions**.
- **How to implement if missing:** Provision the tenant and licenses through Google or an authorized reseller before starting.

2. Source Microsoft 365 / Entra environment (for migration & coexistence)

- **Description:** For migration/coexistence/identity solutions, administrative access to the source **Microsoft 365** tenant and **Microsoft Entra ID** (Global Administrator or the least-privilege roles the solution names).
- **Why required:** Migration, directory sync, SSO federation, and coexistence read from and change the Microsoft side.
- **How to verify:** Confirm access to the M365 admin center and the Entra admin center with the required role.

• **How to implement if missing:** Obtain the role from the M365/Entra tenant owner.

3. DNS control for the routed domains

• **Description:** Ability to create/modify **DNS records** (MX, SPF, DKIM, DMARC, TXT, CNAME) for the domains being migrated, coexisting, or verified.

• **Why required:** Domain verification, mail routing/coexistence, and authentication all depend on DNS changes.

• **How to verify:** Confirm access to the authoritative DNS zone for each domain.

• **How to implement if missing:** Coordinate with whoever controls DNS; stage changes under change control (mail-flow changes are high-impact).

4. Identity & provisioning source of truth

• **Description:** A defined **identity source of truth** (on-prem AD, Entra ID, or an HR/IGA system) and the sync mechanism (e.g. **Google Cloud Directory Sync (GCDS)** or SCIM-based provisioning).

• **Why required:** Consistent users/groups across ecosystems underpin SSO, licensing, and access.

• **How to verify:** Confirm which directory is authoritative and how objects flow into Google.

• **How to implement if missing:** Define the source of truth and provisioning model before configuring SSO or migrating.

5. A pilot scope (OU / group) and change control

• **Description:** A **pilot organizational unit or group** to validate on before fleet-wide rollout, and a **change-control** process for production changes (especially mail flow, DNS, identity, and DLP).

• **Why required:** Every solution follows an **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** model; the pilot proves the design and change control protects production.

• **How to verify:** Confirm a named pilot OU/group exists and a change window/approval path is available.

• **How to implement if missing:** Create a pilot OU/group and agree the change-control process with the owners before making production changes.

6. Security, compliance & governance stakeholders

• **Description:** Engagement with the **security (CISO), legal/eDiscovery, and data-governance** stakeholders whose policies the configuration must satisfy (DLP, retention, Vault/Purview, and any regulatory regime such as ITAR/DFARS/Assured Controls where applicable).

• **Why required:** Data governance, eDiscovery, DLP, and regulated-workload settings must align to enterprise policy and legal requirements, not just technical defaults.

• **How to verify:** Confirm the applicable policies and named stakeholders/approvers.

• **How to implement if missing:** Identify the owners and obtain the policy/compliance requirements before configuring governance controls.

*The deployment model across every Google solution is the same: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback for production changes.*

3. Implementation Guide — Architecture & Design Review Board (ARB) Governance Framework

Follow these steps in order. The board is stood up on paper (charter, decision rights, gates, templates), proven on a **pilot workstream**, then expanded program-wide. Tooling reads and manages the tracker via Microsoft Graph PowerShell; all live-system reads are read-only.

> *Model: Assess to Configure to Pilot to Validate to Expand. You charter the board and open the gates, run one full governance cycle on a pilot workstream's designs, prove decisions are made and recorded without becoming a bottleneck (Step 7 gate), then expand across every workstream.*

Named artifacts you will produce (reuse these names verbatim):

- `ARB Charter - Transformation - Ratified`
- `Decision Rights Matrix - ARB - Program`
- `Submission Template - Design Review - Standard` (one variant per gate)
- `Review Checklist - <Gate Name> - Standard` (one per gate)
- `Architecture Decision Record - <DesignId> - Dated` (an ADR per decision)
- `Decision Register - Transformation ARB - Program-Wide`
- `Standards Baseline - Target Architecture - Approved`
- `Gap Register - Design Drift - Rolling`
- `ARB KPI Set - Governance - Standard`
- ARB membership groups: `ARB - Voting Members - Board` and `ARB - Advisory Reviewers - Panel`

Step 1 — Assess: confirm the mandate and the baseline

1. Confirm the executive sponsor and quote the written mandate (Requirement 2.1) into the charter.
2. Confirm a ratified target architecture exists (Requirement 2.2). Express its key standards as `Standard,ExpectedValue` rows — this becomes the `Standards Baseline - Target Architecture - Approved` that `GWARB-Validation.ps1` reads.
1. If no target architecture is ratified, the board's first act is to adopt one; do not open

conformance gates against an un-ratified standard.

Step 2 — Configure: write the charter and decision rights

1. Draft the `ARB Charter - Transformation - Ratified`: purpose, scope (the five gates), the

vendor/enterprise authority boundary (Requirement 2.4), meeting cadence, and the sponsor mandate.

1. Build the `Decision Rights Matrix - ARB - Program` — for each gate, who submits, who reviews, who

votes, what quorum is, who may approve/reject, and the escalation path for a tie or a blocked decision. Distinguish **voting members** (quorum-bearing) from **advisory reviewers** (no vote).

1. Provision the two membership groups with `GWARB-Set-ScopeTargeting.ps1` (defaults to `-WhatIf`;

review, then re-run with `-Confirm`). Record the returned group IDs in the charter.

Step 3 — Configure: define the five review gates and their templates

1. Adopt the five standard gates: Migration Wave Plan, Identity & Sync Model, Security Configuration,

Data-Migration Pattern, Work-Transformation Approach.

1. For each gate, create a `Submission Template - Design Review - Standard` variant capturing: design

summary, the target-architecture standards it touches, alternatives considered, risks and rollback, the requested decision, and the declared attribute values the board will validate.

1. For each gate, create a `Review Checklist - <Gate Name> - Standard` — the specific conformance

questions reviewers must answer (for example, for Identity & Sync: source of truth, provisioning mechanism, SSO model, coexistence handling).

Step 4 — Configure: stand up the tracker and the register

1. Stand up the submission/decision tracker (Requirement 2.5) with the columns

`GWARB-Discovery.ps1` reads: DesignId, Title, Gate, State, Workstream, DecisionRecordId, Submitted, Owner. A Microsoft List is convenient because the scripts read/write it via Graph.

1. Create the `Decision Register - Transformation ARB - Program-Wide` — one row per ADR, linking

DesignId to its decision, rationale, conditions/exceptions, and owner.

1. Create the `Gap Register - Design Drift - Rolling` — where discovered-vs-approved drift is tracked

and adjudicated.

Step 5 — Pilot: run one governance cycle on a pilot workstream

1. Choose one workstream (for example, the identity workstream) as the pilot scope. Have it submit its real designs through the appropriate gates using the submission templates.

1. Convene the board at the cadence. For each submission, run the gate's review checklist, validate declared attributes against the `Standards Baseline` with `GWARB-Validation.ps1`, and record a decision: Approved, Approved with conditions (log the exception), or Rejected (rework).

1. Write an `Architecture Decision Record - <DesignId> - Dated` for every decision and link it in the register. No pilot design proceeds to build without an ADR.

Step 6 — Run the drift-prevention loop on the pilot

1. `GWARB-Discovery.ps1` — inventory what the pilot workstream has actually submitted/built and its gate state, from the tracker.

1. `GWARB-Validation.ps1` — check declared/built attributes against the approved baseline; every Non-Conformant row is drift.

1. `GWARB-Report.ps1` — produce the Decision Register roll-up and the `Gap Register`, plus the KPI block.

1. `GWARB-Remediation.ps1` (defaults to `-WhatIf`) — open an owned, dated action for each gap; review the plan, then re-run with `-Confirm`. Each gap is closed by rework-to-standard or an approved exception.

Step 7 — Validate: the pilot gate (GATE)

1. Confirm every pilot design has a decision and an ADR; the register and tracker reconcile.

2. Confirm the KPI set is populated: decisions logged, approval rate, open drift, time-in-gate, exception count.

1. Gate: do not expand the board program-wide until the pilot shows decisions are being **made and recorded** and time-in-gate is acceptable (governance is not the bottleneck). If designs pile up undecided, fix cadence/decision-rights before expanding.

Step 8 — Expand: govern every workstream

1. Onboard the remaining workstreams and the delivery partner to the gates and templates; restate the authority boundary so vendor designs come through the board.

1. Run the full loop (Steps 5 to 6) every cadence cycle across all gates; keep the `Decision Register and Gap Register` current.

1. Report the `ARB KPI Set - Governance - Standard` to the sponsor each cycle; a falling open-drift

trend and a stable time-in-gate prove the board is working.

Order of operations

```
Confirm mandate + baseline -> Charter + decision rights + groups -> Gates +
submission/checklist templates -> Tracker + registers -> Pilot governance
cycle -> Drift-prevention loop -> Pilot gate -> Expand program-wide
```

5. Testing Guide — Architecture & Design Review Board (ARB) Governance Framework

Run these after the board is stood up and the pilot cycle is complete (Implementation Step 7), and again each governance cadence cycle. These test governance *effectiveness* — not a technical configuration, but whether the board actually makes, records, and enforces decisions. Use `GWARB-Discovery.ps1` and `GWARB-Report.ps1` for the tracker reads and `GWARB-Validation.ps1` for conformance.

Test 1 — Charter and decision rights are unambiguous

Do: Open the `ARB Charter - Transformation - Ratified` and `Decision Rights Matrix - ARB - Program`. Expected: For every gate you can state who submits, who votes, the quorum number, who may approve/reject, and the escalation path. The vendor/enterprise authority boundary is written. No role or quorum is undefined.

Test 2 — Membership groups exist and match the roster

Do: Run `GWARB-Set-ScopeTargeting.ps1` (review the `-WhatIf` plan) and confirm in Entra. Expected: `ARB - Voting Members - Board` and `ARB - Advisory Reviewers - Panel` exist with the intended members; their IDs are recorded in the charter.

Test 3 — Every gate has a submission template and checklist

Do: Inspect the five gates. Expected: Each of Migration Wave Plan, Identity & Sync Model, Security Configuration, Data-Migration Pattern, and Work-Transformation Approach has a `Submission Template` variant and a `Review Checklist`. No gate is open without both.

Test 4 — Submissions are inventoried and classified

Do: Run `GWARB-Discovery.ps1` against the tracker. Expected: Every submitted design appears with a gate, state, workstream, and owner. No design is missing a gate or owner; any gate outside the standard five is flagged and explained.

Test 5 — Conformance is checked against the approved baseline

Do: Run `GWARB-Validation.ps1` with the `Standards Baseline - Target Architecture - Approved` and the designs' declared attributes. Expected: Each design is Conformant, Conditional (approved exception), or Non-Conformant. The conformance rate is reported; every Non-Conformant row is a drift finding that blocks approval until adjudicated.

Test 6 — Every decision has a record

Do: Run `GWARB-Report.ps1`; review the Decision Register roll-up. Expected: Every gate-stage design has an Architecture Decision Record (HasRecord = yes) or appears in the Gap Register as missing one. No approved/rejected design lacks an ADR.

Test 7 — The Gap Register captures all open drift

Do: Review the `Gap Register - Design Drift - Rolling` from `GWARB-Report.ps1`. Expected: Every Non-Conformant finding and every gate-stage design missing a decision record appears in the Gap Register with an owner. Nothing open is untracked.

Test 8 — Remediation opens owned, dated actions (safely)

Do: Run `GWARB-Remediation.ps1` (default `-WhatIf`) against the Gap Register. Expected: It prints the actions it WOULD open — each with a required board decision, an owner, and a due date — and makes no change. Re-running with `-Confirm` records exactly those actions.

Test 9 — Governance is not the bottleneck

Do: Review time-in-gate and the undecided backlog from the KPI block. Expected: Average time-in-gate is within the target the charter sets, and the count of designs stuck undecided is not growing cycle over cycle.

Pass/fail gate

Gate	Requirement
Pilot pass	Tests 1 to 8 pass for the pilot workstream; every pilot design has a decision and an ADR; Test 9 shows time-in-gate within target
Program pass	Tests 1 to 9 pass across all workstreams; open drift is trending down and no gate is running without a template, checklist, and quorum

6. Troubleshooting Guide — Architecture & Design Review Board (ARB) Governance Framework

Field issues that arise when running the board and its tooling, with the cause and the fix. Governance problems are usually process problems wearing a technical mask — most of these are cured in the charter or cadence, not the scripts.

The board's decisions are ignored

Symptom: A design the board rejected (or approved with conditions) ships anyway. Cause: No enforced authority — the mandate is verbal, or the vendor/enterprise boundary was never written into the SOW. Fix:

Quote the written mandate (Requirement 2.1) in the charter and add the authority boundary (Requirement 2.4) to the SOW or a governance addendum. Escalate the specific breach to the sponsor via the charter's escalation path and record it as an open drift finding so it is visible.

Governance has become the bottleneck

Symptom: Designs pile up undecided; workstreams route around the board to keep to schedule. Cause: Cadence too infrequent, quorum too hard to assemble, or every design forced through a full board. Fix: Increase cadence or add an out-of-band fast-track; shrink the quorum to a tight, senior voting core; delegate standard-conformant, low-risk designs (a wave plan identical to an approved template) to the chair with a lightweight record. Watch time-in-gate (Test 9) as the health signal.

Reviewers disagree and decisions stall on ties

Symptom: The board cannot reach a decision; the same design returns meeting after meeting. Cause: No defined escalation for a tie or a blocked-but-insisted design. Fix: Define escalation in the `Decision Rights Matrix - ARB - Program` before you need it — a named escalation to the sponsor with a decision deadline. Record the escalated decision as an ADR so it is not re-litigated.

`GWARB-Validation.ps1` flags everything as No-Baseline

Symptom: Every row comes back `No-Baseline` and the conformance rate is meaningless. Cause: The `Standard` values in the designs CSV do not match the `Standard` keys in the baseline CSV (a naming or casing mismatch), or the baseline was never populated. Fix: Confirm the `Standards Baseline - Target Architecture - Approved` is ratified and exported as `Standard,ExpectedValue` rows whose names match exactly the `Standard` values in the designs export. Ratify the baseline first (Requirement 2.2) — do not run conformance against an empty standard.

Designs appear with a gate outside the standard five

Symptom: `GWARB-Discovery.ps1` flags designs whose gate is not one of the five standard gates. Cause: A submitter used a free-text gate, or a genuinely new category of design has appeared. Fix: Reclassify the design into its correct gate (its primary risk) in the tracker. If a real new category exists, decide deliberately at the board whether to add a sixth gate — do not let ad-hoc gates accrete, because a gate with no template or checklist cannot make a defensible decision.

Decisions are made but not recorded

Symptom: `GWARB-Report.ps1` shows approved/rejected designs with HasRecord = no. Cause: The board decided in the meeting but no Architecture Decision Record was written. Fix: Make the ADR part of the decision, not a follow-up — write it at the moment of decision and link it in the `Decision Register` before the design leaves the queue. Every unrecorded design shows in the Gap Register until its ADR exists.

Approved exceptions are quietly becoming the norm

Symptom: The exception count in the KPI set keeps rising cycle over cycle. Cause: The baseline no longer matches reality, or gate discipline is slipping so deviations are waved through as exceptions. Fix: Review

whether the standard itself is wrong — if so, update the `Standards Baseline` deliberately through the board rather than absorbing the drift as repeated exceptions. If the standard is right, tighten the gate so exceptions return to being rare and deliberate.

`Connect-MgGraph` fails or the tracker list cannot be read

Symptom: The discovery/remediation scripts error on connect or on reading the Microsoft List. Cause: The Microsoft Graph PowerShell SDK is missing, the required scope was not consented, or the Site/List IDs are wrong. Fix: Install the SDK (`Install-Module Microsoft.Graph -Scope CurrentUser`) and consent the least-privilege scope — `Sites.Read.All` for reads, `Sites.ReadWrite.All` only for remediation writes, `Group.ReadWrite.All` for the group builder. Confirm the Site ID and List ID for the tracker. Or fall back to CSV mode with an export from the tracker.

`GWARB-Remediation.ps1` or `GWARB-Set-ScopeTargeting.ps1` made no changes

Symptom: You ran the script and nothing was created. Cause: Both default to `-WhatIf` (preview) by design — they never change anything until you confirm. Fix: Review the previewed plan, then re-run the same command with `-Confirm` to apply. This is the intended safety behavior, not a fault.

The board outlives its usefulness / becomes a rubber stamp

Symptom: The approval rate is near 100% and meetings are pro forma. Cause: Either the gates no longer catch real architectural choices, or the program has matured past active design. Fix: If designs are genuinely low-risk now, move more to fast-track and reduce cadence. As the transformation closes, retire the board deliberately — fold its standing decisions and baseline into steady-state architecture governance so the decisions outlive the program, rather than letting the forum lapse silently.

7. Best Practices — Architecture & Design Review Board (ARB) Governance Framework

Field-proven recommendations for running an ARB that has authority without becoming the program's bottleneck. Adopt these alongside the Implementation and Testing guides.

Authority

- **Get the mandate in writing before the first meeting.** An ARB with no sponsor backing is a discussion group. The written mandate (Requirement 2.1), quoted in the charter, is what makes "Rejected" stick.
- **Draw the vendor/enterprise boundary explicitly and early.** The enterprise owns the architecture; the vendor executes within it. Put this in the SOW or a governance addendum, not just the charter, so the party you do not manage day to day is still bound to the gates.
- **Never open a conformance gate without a ratified baseline.** A gate with no approved standard behind it produces opinions, not defensible decisions. Ratify the target architecture first.

Decision rights

- **Make quorum small enough to meet and senior enough to bind.** A quorum that cannot assemble weekly

becomes the schedule bottleneck teams route around; one too junior gets overruled later.

- **Separate voting members from advisory reviewers.** Subject-matter experts should review deeply

without diluting the vote; keep the decision with a tight, accountable voting core.

- **Define escalation before you need it.** Write down what happens on a tie or a design the board

blocks but delivery insists on — a named escalation to the sponsor, decided in the charter, not in the heat of a stalled meeting.

Gates and cadence

- **Route each design to exactly one gate — its primary risk.** A design that touches identity and

security goes to the gate for its dominant risk, with the other discipline as an advisory reviewer. One design, one gate, one decision keeps the queue legible.

- **Offer a fast-track for low-risk, standard-conformant designs.** Not everything needs a full board.

A pre-approved pattern (a wave plan identical to an approved template) can be delegated to the chair with a lightweight record, reserving board time for genuine architectural choices.

- **Protect time-in-gate as a first-class KPI.** Governance earns its authority by being fast enough

to trust. If designs pile up undecided, fix cadence or decision rights before adding more gates.

Decision records

- **Write the ADR at the moment of decision, not later.** Record the design, the standards measured,

the decision, the rationale, and any conditions while the discussion is fresh. A register reconstructed after the fact loses exactly the rationale that makes it valuable.

- **Record conditions and exceptions as first-class facts.** "Approved with conditions" is only real

if the conditions are written and later verified. An unrecorded condition is an approval.

- **Treat the register as the program's architectural memory.** Every "why did we do it this way?" a

year from now should be answerable from the `Decision Register - Transformation ARB - Program-Wide`.

Drift prevention

- **Assume drift; hunt for it on a cadence.** Approved designs diverge from reality under delivery

pressure. Run the discover-validate-report-remediate loop every cycle rather than trusting that what was approved is what was built.

- **Adjudicate every drift finding — do not let it sit.** A Non-Conformant finding is either brought

back to standard or formally approved as an exception. Silent, un-adjudicated drift is the exact failure this framework exists to prevent.

- **Watch the exception count as a health signal.** A rising number of approved exceptions means the baseline no longer matches reality — either the standard is wrong (update it deliberately) or discipline is slipping (tighten the gate). Exceptions should be rare and deliberate.

Scope discipline

- **Govern designs, not schedules.** The ARB owns architectural decisions; the PMO owns dates and budget. Blurring the two makes the board slow and resented. Stay in the architecture lane.
- **Prove it on a pilot workstream before program-wide rollout.** One workstream's full governance cycle exposes whether your cadence, quorum, and templates actually work — cheaper to fix on one workstream than across the whole program.
- **Retire the board deliberately at program end.** When the transformation closes, fold the ARB's standing decisions and baseline into the enterprise's steady-state architecture governance rather than letting the forum simply lapse — so the decisions outlive the program.

8. Deliverables Summary — Architecture & Design Review Board (ARB) Governance Framework

Documentation

Document	Purpose
Executive Summary	Business case and what is in the box
1. Solution Overview	What an ARB is, its relationship to the target architecture, the five review gates, the decision-record discipline, drift prevention, and the KPIs
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Google)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered steps from mandate to program-wide governance, with a pilot gate
5. Testing Guide	Governance-effectiveness gates with pass/fail criteria
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations

Named artifacts produced

Artifact	Purpose
`ARB Charter - Transformation - Ratified`	Purpose, scope, authority boundary, cadence, and mandate — the board's constitution
`Decision Rights Matrix - ARB - Program`	Per-gate submit/review/vote/approve rights, quorum, and escalation
`Submission Template - Design Review - Standard`	The form every design is submitted on (one variant per gate)
`Review Checklist - <Gate Name> - Standard`	The conformance questions reviewers answer at each gate
`Architecture Decision Record - <DesignId> - Dated`	One record per decision: design, standards, decision, rationale, conditions
`Decision Register - Transformation ARB - Program-Wide`	The program's architectural memory — every ADR in one register
`Standards Baseline - Target Architecture - Approved`	The approved standards every submission is validated against
`Gap Register - Design Drift - Rolling`	Every open drift finding and missing decision record, with an owner
`ARB KPI Set - Governance - Standard`	Decisions logged, approval rate, open drift, time-in-gate, exception count

Design-Review-Board runbook (outline)

A single operational runbook the secretariat runs each cycle:

1. **Intake** — new submissions land in the tracker against a gate; confirm gate, owner, and that the right submission template is complete.

1. **Prepare** — run `GWARB-Discovery.ps1` to inventory the queue; run `GWARB-Validation.ps1` to pre-check conformance so reviewers arrive with drift already flagged.

1. **Convene** — at cadence, work the queue: apply the gate's `Review Checklist`, decide (Approve / Approve with conditions / Reject), and write the `Architecture Decision Record` for each.

1. **Record** — link every ADR into the `Decision Register`; publish decisions to the workstreams.

2. **Detect drift** — run `GWARB-Report.ps1` for the Decision Register roll-up, the `Gap Register`, and the KPI block.

1. **Remediate** — run `GWARB-Remediation.ps1` (`-WhatIf` first) to open an owned, dated action for each gap; adjudicate every finding as rework-to-standard or an approved exception.

1. **Report** — send the KPI set to the sponsor; confirm time-in-gate and open-drift trends are healthy.

Templates outline (what ships to be filled in)

• **Charter template** — mandate, scope (five gates), authority boundary, membership, quorum, cadence.

• **Decision Rights Matrix template** — a row per gate; columns for submit/review/vote/approve/escalate.

• **Submission templates** — one per gate, each capturing design summary, standards touched, alternatives, risk/rollback, requested decision, and declared attributes.

• **Review checklists** — one per gate, the specific conformance questions for that gate.

• **ADR template** — design, standards measured, decision, rationale, conditions/exceptions, owner, date.

PowerShell scripts

Script	What it does
`GWARB-Discovery.ps1`	Inventories in-flight designs and their gate/state from the tracker (CSV or Microsoft List via Graph); read-only
`GWARB-Validation.ps1`	Checks designs for conformance to the approved standards baseline; flags drift; read-only
`GWARB-Report.ps1`	Produces the Decision Register roll-up, the Gap Register, and the governance KPI set; read-only
`GWARB-Remediation.ps1`	Opens owned, dated remediation actions for each open gap; defaults to `-WhatIf`
`GWARB-Set-ScopeTargeting.ps1`	Creates and seeds the ARB voting and advisory membership groups; defaults to `-WhatIf`

What the buyer receives

• This implementation guide as a branded Word document and PDF.

• All five PowerShell scripts in a single ZIP.

• The full template set (charter, decision-rights matrix, per-gate submission templates and checklists, and the ADR template) plus the Design-Review-Board runbook.

• A repeatable process following the model: Assess to Configure to Pilot to Validate to Expand.

Outcome

A chartered Architecture & Design Review Board with real decision authority — five review gates, recorded decisions, a drift-prevention loop, and governance KPIs — so every major design in the Google Workspace + Gemini transformation is approved against the target architecture, proven on a pilot workstream before it governs the whole program.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

GWARB-Discovery.ps1

Purpose: Reads the Architecture and Design Review Board (ARB) tracking list and emits a design/decision inventory: every submitted design, its review gate, current state, owning workstream, and whether a decision record exists. Feeds the Decision Register. Read-only. Point it at your ARB tracker export (CSV) or, if you keep the tracker as a Microsoft List / SharePoint list, at a Microsoft Graph list so the board's live queue is captured. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\GWARB-Discovery.ps1 -TrackerCsv C:\Temp\arb-tracker.csv Or read a Microsoft List via Graph: powershell -ExecutionPolicy Bypass -File .\GWARB-Discovery.ps1 -SiteId your-site-id -ListId your-list-id Graph mode requires the Microsoft Graph PowerShell SDK and the Sites.Read.All scope (read-only). Tracker CSV columns: DesignId,Title,Gate,State,Workstream,DecisionRecordId,Submitted,Owner EXPECTED RESULTS A per-design table (id, gate, state, workstream, has-decision-record) and counts by gate and state. Rows in a review gate with no decision record are flagged as open items. No system is changed. Pass = the inventory reconciles with the board's tracker and no submitted design is missing a gate or owner. .NOTES White-label — the repeatable way. Read-only where it reads live systems. #> [CmdletBinding(DefaultParameterSetName = 'Csv')] param([Parameter(ParameterSetName = 'Csv')][string]$TrackerCsv, [Parameter(ParameterSetName = 'Graph', Mandatory)][string]$SiteId, [Parameter(ParameterSetName = 'Graph', Mandatory)][string]$ListId, [string]$CsvPath)

How to run: powershell -ExecutionPolicy Bypass -File .\GWARB-Discovery.ps1 -TrackerCsv C:\Temp\arb-tracker.csv Or read a Microsoft List via Graph: powershell -ExecutionPolicy Bypass -File .\GWARB-Discovery.ps1 -SiteId your-site-id -ListId your-list-id Graph mode requires the Microsoft Graph PowerShell SDK and the Sites.Read.All scope (read-only). Tracker CSV columns: DesignId,Title,Gate,State,Workstream,DecisionRecordId,Submitted,Owner EXPECTED RESULTS A per-design table (id, gate, state, workstream, has-decision-record) and counts by gate and state. Rows in a review gate with no decision record are flagged as open items. No system is changed. Pass = the inventory reconciles with the board's tracker and no submitted design is missing a gate or owner. .NOTES White-label — the repeatable way. Read-only where it reads live systems. #> [CmdletBinding(DefaultParameterSetName = 'Csv')] param([Parameter(ParameterSetName = 'Csv')][string]$TrackerCsv, [Parameter(ParameterSetName = 'Graph', Mandatory)][string]$SiteId, [Parameter(ParameterSetName = 'Graph', Mandatory)][string]$ListId, [string]$CsvPath)

GWARB-Remediation.ps1

Purpose: Turns the Gap Register (from GWARB-Report.ps1) into tracked remediation actions: for each open drift or missing-decision-record finding it drafts an action item (owner, due date, required board decision) and, where you keep the ARB tracker as a Microsoft List, creates the item via Microsoft Graph. Defaults to -WhatIf so nothing is created until you review the plan and re-run with -Confirm. This script never changes a target-system configuration; it only records the governance actions the board has decided to pursue. HOW TO RUN Preview (default, no change): powershell -ExecutionPolicy Bypass -File .\GWARB-Remediation.ps1 -GapCsv C:\Temp\gap-register.csv Create the action items in the ARB Microsoft List (after review): powershell -ExecutionPolicy Bypass -File .\GWARB-Remediation.ps1 -GapCsv C:\Temp\gap-register.csv -SiteId your-site-id -ListId your-actions-list-id -Confirm Graph mode requires the Microsoft Graph PowerShell SDK and the Sites.ReadWrite.All scope. GapCsv columns:

DesignId,Gate,State,Gap,Owner EXPECTED RESULTS In -WhatIf: the remediation actions that WOULD be opened, with owner and due date — no change. With -Confirm: a CSV action plan is written, and (in Graph mode) one list item is created per finding. Pass = every open gap has an owned, dated remediation action. .NOTES White-label — the repeatable way. Read-only where it reads live systems. #> [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([Parameter(Mandatory)][string]$GapCsv, [string]$SiteId, [string]$ListId, [int]$DueInDays = 10, [string]$OutDir = '.')

How to run: Preview (default, no change): powershell -ExecutionPolicy Bypass -File .\GWARB-Remediation.ps1 -GapCsv C:\Temp\gap-register.csv Create the action items in the ARB Microsoft List (after review): powershell -ExecutionPolicy Bypass -File .\GWARB-Remediation.ps1 -GapCsv C:\Temp\gap-register.csv -SiteId your-site-id -ListId your-actions-list-id -Confirm Graph mode requires the Microsoft Graph PowerShell SDK and the Sites.ReadWrite.All scope. GapCsv columns: DesignId,Gate,State,Gap,Owner EXPECTED RESULTS In -WhatIf: the remediation actions that WOULD be opened, with owner and due date — no change. With -Confirm: a CSV action plan is written, and (in Graph mode) one list item is created per finding. Pass = every open gap has an owned, dated remediation action. .NOTES White-label — the repeatable way. Read-only where it reads live systems. #> [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([Parameter(Mandatory)][string]$GapCsv, [string]$SiteId, [string]$ListId, [int]$DueInDays = 10, [string]$OutDir = '.')

GWARB-Report.ps1

Purpose: Joins the design inventory (from GWARB-Discovery.ps1) with the conformance results (from GWARB-Validation.ps1) into a single Decision Register plus a Gap Register of open drift and missing decision records. Computes the governance KPI set: decisions logged, approval rate, open drift, average time-in-gate, and exception count. Read-only. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\GWARB-Report.ps1 -InventoryCsv C:\Temp\inventory.csv -ConformanceCsv C:\Temp\conformance.csv -OutDir C:\Temp\arb-report InventoryCsv = output of GWARB-Discovery.ps1 (DesignId,Gate,State,Workstream,HasRecord,Owner) ConformanceCsv = a per-design roll-up (DesignId,Status) — worst status per design from GWARB-Validation.ps1 Optional: add a Submitted and Decided column set for time-in-gate. No live-system scopes required. EXPECTED RESULTS A Decision Register CSV, a Gap Register CSV, and a printed KPI block. Pass = the registers are produced and the Gap Register lists every open drift item and every gate-stage design missing a decision record. No system is changed. .NOTES White-label — the repeatable way. Read-only where it reads live systems. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$InventoryCsv, [string]$ConformanceCsv, [string]$OutDir = '.')

How to run: powershell -ExecutionPolicy Bypass -File .\GWARB-Report.ps1 -InventoryCsv C:\Temp\inventory.csv -ConformanceCsv C:\Temp\conformance.csv -OutDir C:\Temp\arb-report InventoryCsv = output of GWARB-Discovery.ps1 (DesignId,Gate,State,Workstream,HasRecord,Owner) ConformanceCsv = a per-design roll-up (DesignId,Status) — worst status per design from GWARB-Validation.ps1 Optional: add a Submitted and Decided column set for time-in-gate. No live-system scopes required. EXPECTED RESULTS A Decision Register CSV, a Gap Register CSV, and a printed KPI block. Pass = the registers are produced and the Gap Register lists every open drift item and every gate-stage design missing a decision record. No system is changed. .NOTES White-label — the repeatable way. Read-only where it reads live systems. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$InventoryCsv, [string]$ConformanceCsv, [string]$OutDir = '.')

GWARB-Set-ScopeTargeting.ps1

Purpose: Creates (idempotently) the security groups that scope the Architecture and Design Review Board: 'ARB - Voting Members - Board' (quorum-bearing decision-makers) and 'ARB - Advisory Reviewers - Panel' (subject-matter reviewers with no vote). Optionally seeds each group from a CSV of user principal names so the board's roster is ready for the tracker and notifications. Defaults to -WhatIf so nothing is created until you review the plan. HOW TO RUN Preview (default, no change): powershell -ExecutionPolicy Bypass -File .\GWARB-Set-ScopeTargeting.ps1 Create groups and seed rosters (after review): powershell -ExecutionPolicy Bypass -File .\GWARB-Set-ScopeTargeting.ps1 -VotingMembersCsv C:\Temp\voting.csv -AdvisoryMembersCsv C:\Temp\advisory.csv -Confirm Each CSV needs a UserPrincipalName column. Requires the Microsoft Graph PowerShell SDK and the Group.ReadWrite.All and User.Read.All scopes. EXPECTED RESULTS In -WhatIf: the groups and seed actions that WOULD run — no change. With -Confirm: both groups exist (created if missing, reused if present) and are seeded from any CSV supplied; group object IDs are printed. Pass = both ARB groups exist with the intended rosters. Record the IDs in the ARB Charter. .NOTES White-label — the repeatable way. Read-only where it reads live systems. #> [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'Medium')] param([string]$VotingMembersCsv, [string]$AdvisoryMembersCsv)

How to run: Preview (default, no change): powershell -ExecutionPolicy Bypass -File .\GWARB-Set-ScopeTargeting.ps1 Create groups and seed rosters (after review): powershell -ExecutionPolicy Bypass -File .\GWARB-Set-ScopeTargeting.ps1 -VotingMembersCsv C:\Temp\voting.csv -AdvisoryMembersCsv C:\Temp\advisory.csv -Confirm Each CSV needs a UserPrincipalName column. Requires the Microsoft Graph PowerShell SDK and the Group.ReadWrite.All and User.Read.All scopes. EXPECTED RESULTS In -WhatIf: the groups and seed actions that WOULD run — no change. With -Confirm: both groups exist (created if missing, reused if present) and are seeded from any CSV supplied; group object IDs are printed. Pass = both ARB groups exist with the intended rosters. Record the IDs in the ARB Charter. .NOTES White-label — the repeatable way. Read-only where it reads live systems. #> [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'Medium')] param([string]$VotingMembersCsv, [string]$AdvisoryMembersCsv)

GWARB-Validation.ps1

Purpose: Compares each design's declared attributes against the ARB-approved standards baseline (the target-architecture standards the board ratified) and reports per-design conformance: conformant, conditional (has an approved exception), or non-conformant (drift). Each non-conformance is a gate finding the board must adjudicate before approval. Read-only. The baseline and each design's attributes are supplied as data — this script never invents a standard; it enforces the ones your board has approved. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\GWARB-Validation.ps1 -DesignsCsv C:\Temp\designs.csv -BaselineCsv C:\Temp\baseline.csv Optional approved-exceptions register: ... -ExceptionsCsv C:\Temp\exceptions.csv BaselineCsv columns: Standard,ExpectedValue DesignsCsv columns: DesignId,Standard,ActualValue ExceptionsCsv columns: DesignId,Standard (an ARB-approved deviation) No live-system scopes are required; runs entirely on the supplied registers. EXPECTED RESULTS A per-design, per-standard conformance table and a conformance rate. Pass = every design at or past its gate is conformant or has an approved exception; any Non-Conformant row is drift that blocks approval until adjudicated. No system is changed. .NOTES White-label — the repeatable way. Read-only where it reads live systems. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$DesignsCsv, [Parameter(Mandatory)][string]$BaselineCsv, [string]$ExceptionsCsv)

How to run: powershell -ExecutionPolicy Bypass -File .\GWARB-Validation.ps1 -DesignsCsv C:\Temp\designs.csv -BaselineCsv C:\Temp\baseline.csv Optional approved-exceptions register: ... -ExceptionsCsv C:\Temp\exceptions.csv BaselineCsv columns: Standard,ExpectedValue DesignsCsv columns: DesignId,Standard,ActualValue ExceptionsCsv columns: DesignId,Standard (an ARB-approved deviation) No live-system scopes are required; runs entirely on the supplied registers. EXPECTED RESULTS A per-design, per-standard conformance table and a conformance rate. Pass = every design at or past its gate is conformant or has an approved exception; any Non-Conformant row is drift that blocks approval until adjudicated. No system is changed. .NOTES White-label — the repeatable way. Read-only where it reads live systems. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$DesignsCsv, [Parameter(Mandatory)][string]$BaselineCsv, [string]$ExceptionsCsv)